

AI 에이전트

전문가 양성 과정

제어문



Contents

1. 조건문

- if
- match - case

2. 반복문

- for
- while

3. 함수

- 함수 기초
- 가변인자
- Scope
- 함수 응용
- 내장 함수
- lambda

제 1장. 조건문

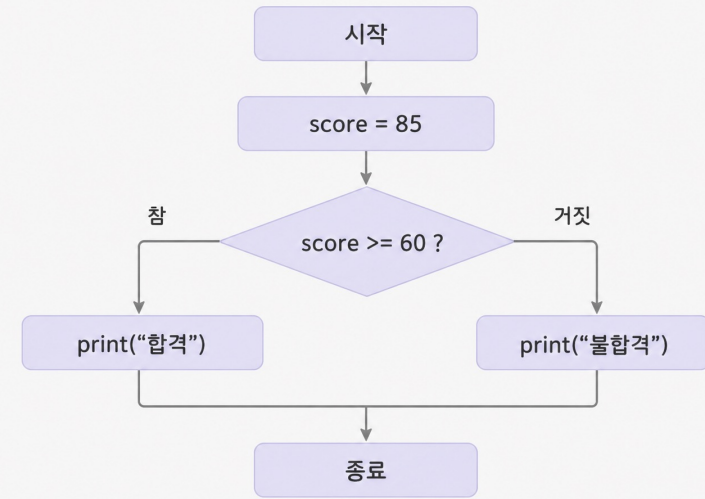
1-1 if

if

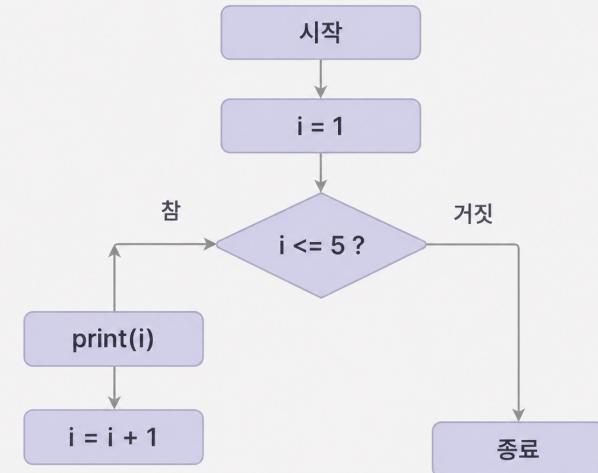
• 제어문

- 프로그램의 실행 흐름을 조절하는 문법
- 프로그램은 기본적으로 위에서 아래로 한 줄씩 실행
- 하지만 실제 프로그램은 항상 같은 순서로만 동작하지 않음
- 특정 상황에 따라 코드를 선택적으로 실행하거나 (분기/조건 처리) 계속해서 실행하는 제어가 필요 (반복문)

조건문 (if / else)



반복문 (while)



if

- 조건문
 - 조건에 따라 다른 코드를 실행하는 문법
 - 조건이 참이면 특정 코드 실행
 - 조건이 거짓이면 실행하지 않거나, 다른 코드 실행
- 기본 문법
 - if 뒤에 조건식 작성
 - 조건식 뒤에 콜론 : 붙이기
 - 실행할 코드는 들여쓰기

```
if 조건문:  
    실행할_코드1  
    실행할_코드2  
    ...  
  
else:  
    실행할_코드1  
    실행할_코드2  
    ...
```

```
age = 20  
  
if age >= 19:  
    print("성인입니다.")  
else:  
    print("미성년자입니다.")
```

if

- 조건문
- 들여쓰기
 - Python은 들여쓰기로 코드의 포함 관계를 판단

```
age = 20

if age >= 19:
    print("성인입니다.")
    print("입장 가능합니다.")
else:
    print("미성년자입니다.")
    print("입장 불가능합니다.")

print("프로그램 종료")
```

```
has_coupon = True

if has_coupon:
    print("할인을 적용합니다.")
else:
    print("할인을 적용하지 않습니다.")
```

if

- 조건문
 - elif
 - 조건이 여러 개일 때는 elif 사용
 - 먼저 참이 되는 조건의 코드만 실행
 - if는 처음 조건
 - elif는 추가 조건
 - else는 나머지 모든 경우

```
score = 85

if score >= 90:
    print("A등급")
elif score >= 80:
    print("B등급")
elif score >= 70:
    print("C등급")
else:
    print("D등급")
```


if

- 간단 실습
 - `input()` 함수를 사용해 사용자로 부터 숫자를 입력 받고, `num`이라는 변수에 할당하시오.
 - 조건문을 통해 `num`의 홀수 or 짝수 여부를 출력하시오.
 - 홀수인 경우 : '홀수입니다.'
 - 짝수인 경우 : '짝수입니다.'

if

- 중첩 조건문
 - if 문 안에 또 다른 if 문을 넣을 수 있다.

```
is_logged_in = True
is_admin = False

if is_logged_in:
    if is_admin:
        print("관리자 페이지로 이동합니다.")
    else:
        print("일반 사용자 페이지로 이동합니다.")
else:
    print("로그인이 필요합니다.")
```

1-2 match - case

match - case

- if 조건문의 단점
 - 하나의 변수 값에 따라 분기하는 경우에는 코드가 다소 길어질 수 있음
 - grade ==이 반복되어 읽기에 깔끔하지 않음

```
grade = 'B'

if grade == 'A':
    print("탁월한 성적입니다.")
elif grade == 'B':
    print("우수한 성적입니다.")
elif grade == 'C':
    print("보통입니다.")
else:
    print("노력이 필요합니다.")
```



```
grade = 'B'
match grade:
    case 'A':
        print("탁월한 성적입니다.")
    case 'B':
        print("우수한 성적입니다.")
    case 'C':
        print("보통입니다.")
    case _:
        print("노력이 필요합니다.")
```

match - case

- if 조건문의 단점
 - match 뒤에 비교할 변수 작성, case 뒤에 해당 변수와 비교할 값 작성
 - 어떤 case에도 일치하지 않으면 case _에 속한 문장을 수행

```
match 변수:  
    case 패턴1:  
        수행할_코드1  
        ...  
    case 패턴2:  
        수행할_코드2  
        ...  
    case _:  
        수행할_코드3  
        ...
```

제 2장.

반복문

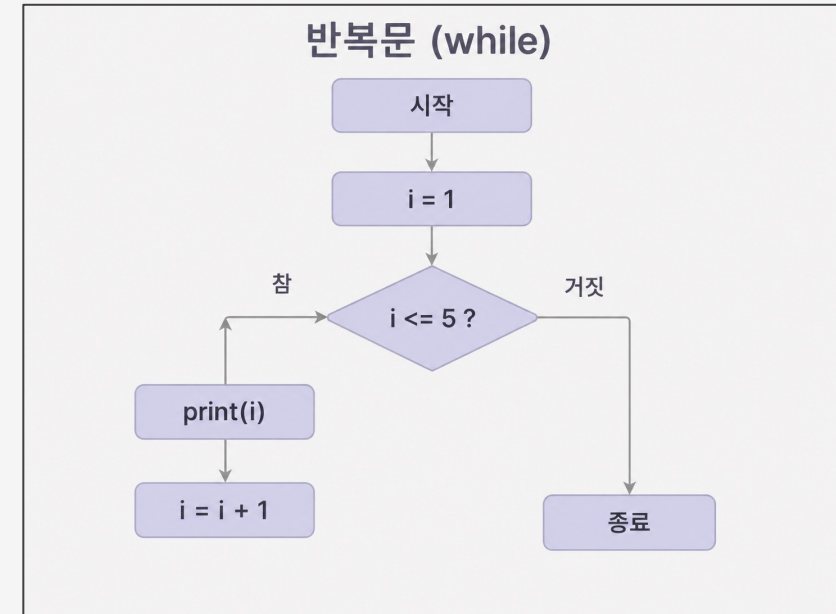
2-1 for

for

- 반복문

- 같은 코드를 여러 번 실행하는 문법

- 학생 점수 여러 개를 출력할 때
- 상품 목록을 하나씩 확인할 때
- 파일 목록을 하나씩 처리할 때
- 사용자 메시지를 순서대로 읽을 때
- 같은 작업을 정해진 횟수만큼 실행할 때



for

- for 문법

1. fruits에서 "apple"을 꺼내 fruit에 저장
2. print(fruit) 실행
3. fruits에서 "banana"를 꺼내 fruit에 저장
4. print(fruit) 실행
5. fruits에서 "orange"를 꺼내 fruit에 저장
6. print(fruit) 실행

```
for 변수 in 반복할_대상:  
    반복해서 실행할 코드
```

```
fruits = ["apple", "banana", "orange"]  
  
for fruit in fruits:  
    print(fruit)
```

for

- 반복할 수 있는 대상 (iterable)
 - 리스트
 - 튜플
 - 문자열
 - range()
 - enumerate()
 - 파일
 - ...

```
for x in [10, 20, 30]:  
    print(x) # 10, 20, 30
```

```
for x in (10, 20, 30):  
    print(x) # 10, 20, 30
```

```
for ch in "Python":  
    print(ch) # P, y, t, h, o, n
```

```
for x in {"name": "Tom", "age": 20} :  
    print(x) # name, age
```

```
for i in range(5):  
    print(i) # 0, 1, 2, 3, 4
```

```
with open("data.txt", "r", encoding="utf-8") as f:  
    for line in f:  
        print(line)
```

for

- for 문법
 - 리스트의 요소값이 튜플
 - 각각의 요소가 자동으로 (x, y)에 대입

```
dataset = [(1, 2), (3, 4), (5, 6)]  
  
for x, y in dataset:  
    print(x + y)
```

for

- 간단 실습
 - 총 5명의 학생이 시험을 보았는데 시험 점수가 60점 이상이면 합격이고 그렇지 않으면 불합격이다.
 - 각 학생의 시험 결과가 합격인지, 불합격인지 결과를 출력하시오.

```
marks = [90, 25, 67, 45, 80]

# 출력 예시
"1번 학생은 90점으로 합격입니다."
"2번 학생은 25점으로 불합격입니다."
...
```

for

- for와 range()
 - 정해진 횟수만큼 반복할 때는 range() 사용
 - range() 문법

```
range(끝)  
range(시작, 끝)  
range(시작, 끝, 간격)
```

```
for i in range(5):  
    print(i)
```

```
for i in range(1, 6):  
    print(i)
```

```
for i in range(1, 10, 2):  
    print(i)
```

```
for i in range(3):  
    print("안녕하세요")
```

for

- for와 문자열
 - 문자열을 반복 시 글자 하나씩 꺼낼 수 있다

```
text = "Python"

for char in text:
    print(char)
```

for

- break & continue & pass
 - **break** : 반복문을 즉시 종료할 때 사용
 - **continue** : 현재 반복을 건너뛰는 때 사용 (반복문 전체를 멈추지는 않음)
 - **pass** : 아무것도 하지 않고 넘어갈 때 사용

```
import keyword
print(keyword.kwlist)

"""
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await',
'break', 'class', 'continue', 'def', 'del', 'elif', 'else',
'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in',
'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return',
'try', 'while', 'with', 'yield']
"""
```

for

- break & continue
 - break : 반복문을 즉시 종료할 때 사용

```
numbers = [1, 2, 3, 5, 7, 8, 9]

for number in numbers:
    if number % 2 == 0:
        print(f"첫 번째 짝수 발견: {number}")
        break
```

```
numbers = [1, 2, 3, 5, 7, 8, 9]

for number in numbers:
    print(number)

    if number % 2 == 0 :
        break
```


for

- break & continue
 - continue : 현재 반복을 건너뛰는 때 사용
(반복문 전체를 멈추지는 않음)
 - 점수가 없는 학생은 건너뛰기
 - 품질 상품은 주문 처리에서 제외하기
 - 빈 문자열은 처리하지 않기
 - 잘못된 데이터는 계산에서 제외하기

```
students = [  
    {"name": "철수", "score": 90},  
    {"name": "영희", "score": None},  
    {"name": "민수", "score": 75},  
    {"name": "수진", "score": None},  
]  
  
for student in students:  
    name = student["name"]  
    score = student["score"]  
  
    if score is None:  
        print(f"{name}: 결석으로 채점 제외")  
        continue  
  
    if score >= 60:  
        print(f"{name}: 통과")  
    else:  
        print(f"{name}: 재시험")
```

for

- `pass`
 - 문법상 코드 블록이 필요하지만, 아직 작성할 코드가 없을 때 사용
 - Python에서는 `if`, `for`, `while`, `def`, `class` 아래에 반드시 실행할 코드가 있어야 함
 - 아직 코드를 작성하지 않았지만 구조만 잡고 싶을 때 `pass`를 사용

```
for number in range(5):  
    pass
```

```
numbers = [1, 2, 3, 4, 5]  
  
for number in numbers:  
    if number % 2 == 0:  
        pass  
    else:  
        print(number)
```

for

- pass vs continue
 - pass와 continue는 둘 다 넘어간다는 느낌이 있지만 동작이 다름
 - **pass** : 아무것도 하지 않을 뿐. 아래의 print(number)는 그대로 실행
 - **continue** : 현재 반복의 나머지 코드를 건너 뛴. 그래서 2는 출력되지 않음

```
numbers = [1, 2, 3]

for number in numbers:
    if number == 2:
        pass

    print(number)
```

```
numbers = [1, 2, 3]

for number in numbers:
    if number == 2:
        continue

    print(number)
```

for

- enumerate
 - 리스트를 반복할 때 순서 번호와 값을 함께 꺼내고 싶을 때 사용
 - 기본 인덱스는 0부터 시작, start= 를 사용하면 x번부터 출력 가능

```
fruits = ["apple", "banana", "orange"]
```

```
for index, fruit in enumerate(fruits):  
    print(index, fruit)
```

```
# 0 apple  
# 1 banana  
# 2 orange
```

```
fruits = ["apple", "banana", "orange"]
```

```
for index, fruit in enumerate(fruits, start=1):  
    print(f"{index}. {fruit}")
```

```
# 1. apple  
# 2. banana  
# 3. orange
```

for

- zip
 - 여러 리스트를 나란히 묶어서 반복할 때 사용

```
names = ["철수", "영희", "민수"]
scores = [90, 85, 70]

for name, score in zip(names, scores):
    print(f"{name}: {score}점")

# 철수: 90점
# 영희: 85점
# 민수: 70점
```

for

- zip
 - 여러 리스트를 나란히 묶어서 반복할 때 사용

```
products = ["삼각김밥", "바나나우유", "도시락", "콜라"]
prices = [1200, 1800, 4500, 1500]
quantities = [3, 2, 1, 4]

for product, price, quantity in zip(products, prices, quantities):
    sales = price * quantity
    print(f"{product}의 매출은 {sales}원입니다.")

# 삼각김밥의 매출은 3600원입니다.
# 바나나우유의 매출은 3600원입니다.
# 도시락의 매출은 4500원입니다.
# 콜라의 매출은 6000원입니다.
```

for

- zip
 - zip()은 길이가 다르면 짧은 쪽에 맞춰서 반복

```
names = ["철수", "영희", "민수"]
scores = [90, 85]

for name, score in zip(names, scores):
    print(f"{name}: {score}점")

# 철수: 90점
# 영희: 85점
```

for

- 리스트 컴프리헨션 (list comprehension)
 - 반복문을 사용해 리스트를 만드는 짧은 문법

```
numbers = [1, 2, 3, 4, 5]
squares = []

for number in numbers:
    squares.append(number ** 2)

print(squares)
```



```
numbers = [1, 2, 3, 4, 5]
squares = [number ** 2 for number in numbers]
print(squares)
```


for

- 리스트 컴프리헨션 (list comprehension)
 - 반복문을 사용해 리스트를 만드는 짧은 문법

[표현식 for 변수 in 반복할_대상]

```
numbers = [1, 2, 3, 4, 5]
squares = [number ** 2 for number in numbers]
print(squares)
```

for

- 리스트 컴프리헨션 (list comprehension)
 - 조건이 있는 리스트 컴프리헨션
 - 리스트 컴프리헨션은 짧고 읽기 쉬울 때 사용하기 좋음

[표현식 `for` 변수 `in` 반복할_대상 `if` 조건식]

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
even_numbers = [number for number in numbers if number % 2 == 0]
```

```
print(even_numbers)
```

for

- 리스트 컴프리헨션 (list comprehension)
 - 조건이 있는 리스트 컴프리헨션
 - 리스트 컴프리헨션은 짧고 읽기 쉬울 때 사용하기 좋음

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
result = [x * 2 if x > 10 else x + 1 for x in numbers if x % 2 == 0]
```

2-2 while

while

- 조건이 참인 동안 계속 반복하는 문법
 - 조건이 True면 반복, False면 멈춤 (탈출)
 1. count가 1인지 확인
 2. count <= 3 이므로 True
 3. print(count) 실행
 4. count를 2로 변경
 5. count가 2인지 확인
 6. count <= 3 이므로 True
 7. print(count) 실행
 8. count를 3으로 변경
 9. count가 3인지 확인
 10. count <= 3 이므로 True
 11. print(count) 실행
 12. count를 4로 변경
 13. count가 4인지 확인
 14. count <= 3 이므로 False
 15. 반복 종료

`while` 조건식:
반복해서 실행할 코드

```
count = 1

while count <= 3:
    print(count)
    count = count + 1
```

while

- 조건이 참인 동안 계속 반복하는 문법
 - 조건이 True면 반복, False면 멈춤 (탈출)

```
while 조건식:  
    반복해서 실행할 코드
```

이 부분을 지우면 어떻게 될까?

```
count = 1  
  
while count <= 3:  
    print(count)  
    count = count + 1
```



while

- while과 break
 - 조건이 True면 반복, False면 멈춤 (탈출)

```
while True:
    user_input = input("종료하려면 q를 입력하세요: ")

    if user_input == "q":
        print("프로그램을 종료합니다.")
        break

print(f"입력한 값: {user_input}")
```

while

- while과 for

구분	for문	while문
기준	반복할 대상이 있을 때	조건이 참인 동안
주로 쓰는 상황	리스트, 문자열, range 반복	언제 끝날지 조건으로 판단
예시	학생 목록 하나씩 출력	사용자가 q를 입력할 때까지 반복

while

- while과 for
 - 몇 번 입력할지 미리 알 수 없음 (사용자가 올바른 비밀번호를 입력할 때까지 반복해야 함)
 - 이런 경우는 while문이 더 자연스러움

```
scores = [90, 80, 70]
```

```
for score in scores:  
    print(score)
```

```
user_input = ""
```

```
while not user_input.isdigit():  
    user_input = input("숫자를 입력하세요: ")  
  
    number = int(user_input)  
  
    print(f"입력한 숫자: {number}")
```

제 3장.

함수

3-1 함수 기초

함수 기초

- 자주 사용하는 코드를 이름 붙여 묶어둔 것

```
stock_name_1 = "삼성전자"  
current_price_1 = 73500  
buy_price_1 = 70000  
quantity_1 = 10
```

```
profit_amount_1 = (current_price_1 - buy_price_1) * quantity_1  
print(f"{stock_name_1} 평가 손익: {profit_amount_1}원")
```

```
stock_name_2 = "SK하이닉스"  
current_price_2 = 118000  
buy_price_2 = 120000  
quantity_2 = 3
```

```
profit_amount_2 = (current_price_2 - buy_price_2) * quantity_2  
print(f"{stock_name_2} 평가 손익: {profit_amount_2}원")
```

함수 기초

- 자주 사용하는 코드를 이름 붙여 묶어둔 것

```
def show_profit_message(stock_name, buy_price, current_price, quantity):  
    profit_amount = (current_price - buy_price) * quantity  
    print(f"{stock_name} 평가 손익: {profit_amount}원")
```

```
show_profit_message("삼성전자", 70000, 73500, 10)  
show_profit_message("SK하이닉스", 120000, 118000, 3)
```

함수 기초

- 자주 사용하는 코드를 이름 붙여 묶어둔 것

```
print("hello world")
```

```
len([1, 2, 3, 4, 5])
```

함수 기초

- 자주 사용하는 코드를 이름 붙여 묶어둔 것

len([1, 2, 3, 4, 5])



```
numbers = [1, 2, 3, 4, 5]
```

```
count = 0
```

```
for i in numbers:
```

```
    count += 1
```

```
print(count)
```

함수 기초

- 자주 사용하는 코드를 이름 붙여 묶어둔 것
 - 프로그래밍을 하다 보면 똑같은 내용을 반복해서 작성하는 경우 발생
 - 반복되는 부분이 있을 경우, '반복적으로 사용되는 가치 있는 부분'을 한 덩치로 묶어 함수로 작성
- 함수 활용의 장점
 - 필요할 때마다 함수 이름을 불러서 실행할 수 있음
 - 같은 코드를 반복해서 쓰지 않아도 됨
 - 프로그램을 작은 단위로 나누어 관리할 수 있음

```
print("안녕하세요!")  
print("환영합니다.")
```

```
def say_hello():  
    print("안녕하세요!")  
    print("환영합니다.")
```

```
say_hello()  
say_hello()
```


함수 기초

- 함수의 구조
 - def는 함수를 만들 때 사용하는 키워드
 - 함수 이름 뒤에는 소괄호 ()와 콜론 : 을 붙인다
 - 함수 안에서 실행할 코드는 들여쓰기
 - 매개변수 (parameter)
 - : 함수가 값을 받을 때 사용하는 변수
 - 인자 (argument)
 - : 함수를 호출할 때 실제로 전달하는 값

```
def 함수_이름(매개변수):  
    수행할_코드1  
    수행할_코드2  
    ...
```

```
def add(a, b):  
    print(a + b)
```

함수 기초

- 함수의 구조
 - 함수는 만들어도 자동 실행되지 않음, 반드시 호출(실행) 코드가 있어야 함

```
# 함수 작성
def add(a, b):
    print(a + b)

# 함수 호출
add(5, 3)      # 8
add(12, 13)   # 25
```

함수 기초

- 아래 코드 실행 결과는?

```
def add(a, b):  
    print(a + b)  
  
print("함수 호출 전")  
  
add(15, 13)  
  
print("함수 호출 후")
```

함수 기초

- **return**
 - 함수의 결과를 함수 밖으로 반환하는 문법
 - return하지 않은 함수는 None 반환

```
import keyword
print(keyword.kwlist)

"""
['False', 'None', 'True', 'and', 'as', 'assert',
'async', 'await', 'break', 'class', 'continue', 'def',
'del', 'elif', 'else', 'except', 'finally', 'for',
'from', 'global', 'if', 'import', 'in', 'is', 'lambda',
'nonlocal', 'not', 'or', 'pass', 'raise', 'return',
'try', 'while', 'with', 'yield']
"""
```

```
def add(a, b):
    return a + b

total = add(10, 20)

print(total) # 30
```

함수 기초

- `return`
 - 함수의 결과를 함수 밖으로 반환하는 문법
 - ✱ `print` : 터미널에 출력하기 위한 함수

```
def add(a, b):  
    return a + b  
  
total = add(10, 20)  
  
print(total) # 30
```

```
def add(a, b):  
    print(a + b)  
  
total = add(10, 20)  
  
print(total) # None
```

👉 `return`을 사용하면 결과를 다시 사용 가능

함수 기초

- `return`
 - `return`이 실행되면 함수는 즉시 종료
 - `return` 아래에 있는 코드는 실행되지 않음

```
def get_result(a, b):  
    return a + b  
    return a * b  
  
result = get_result(3, 5)  
  
print(result) # 8
```

👉 여러 값을 반환하고 싶다면?

함수 기초

- `return`
 - `return` 을 여러 번 쓰는 것이 아니라, 한 번의 `return` 으로 여러 값을 함께 반환
 - 튜플로 묶여서 반환

```
def get_result(a, b):  
    return a + b  
    return a * b  
  
result = get_result(3, 5)  
  
print(result) # 8
```

```
def get_result(a, b):  
    return a + b , a * b  
  
result = get_result(3, 5)  
  
print(result) # (8, 15)
```

함수 기초

- 파라미터가 없는 함수
 - 파라미터가 없는 함수가 가능할까?

```
def say():  
    return 'Hi'
```


함수 기초

- 위치 인자 vs 키워드 인자
 - 함수 호출 시 인자는 위치에 따라 함수에 전달 (위치 인자)
 - 키워드 인자 : 변수 이름으로 특정 인자를 전달
 - 어떤 값이 어떤 매개변수에 들어가는지 명확
 - 인자의 순서를 덜 헷갈림

```
def say_hello(name, age, language):  
    print(name)  
    print(age)  
    print(language)  
  
say_hello("Jay", 25, "Python")  
say_hello(name="Jay", age=25, language="Python")  
say_hello(language="Python", name="Jay", age=25)
```

함수 기초

- 위치 인자 vs 키워드 인자
 - 키워드 인자 다음 위치 인자를 활용할 순 없음

```
def say_hello(name, age, language):  
    print(name)  
    print(age)  
    print(language)  
  
say_hello(name="Jay", 25, "Python")
```

👉 에러 발생

함수 기초

- 기본값이 있는 파라미터
 - 매개변수에 기본값을 줄 수 있음
 - 값을 전달하지 않으면 기본값 사용 / 값을 전달하면 전달한 값을 사용

```
def say_hello(name="손님"):
    print(f"안녕하세요, {name}님!")

say_hello()
say_hello("Jay")
```

함수 기초

- 실행될까? 결과는 어떻게 출력될까?

```
def say_hello(name, age = 30, language = "Python"):
    print(name)
    print(age)
    print(language)

say_hello(name="Jay")
say_hello(name="Amy", age=25)
```

3-2 가변 인자

가변 인자

- 함수에 전달할 값의 개수가 매번 달라질 수 있음
- 이때 사용하는 문법이 가변 인자
- *args, **kwargs 를 주로 사용

```
print('hello', 'world')
```

가변 인자

- `*args` : 여러 개의 위치 인자를 받을 때 사용
 - 일반 함수는 인자 개수가 정해져 있음
 - 숫자 2개가 아니라 3개, 4개, 5개를 더하고 싶다면?
 - 매번 함수를 새로 만들 수는 없음

```
def add(a, b):  
    return a + b  
  
print(add(10, 20))
```

```
add(10, 20, 30)  
add(10, 20, 30, 40)  
add(10, 20, 30, 40, 50)
```

가변 인자

- `*args` : 여러 개의 위치 인자를 받을 때 사용
 - 일반 함수는 인자 개수가 정해져 있음
 - 숫자 2개가 아니라 3개, 4개, 5개를 더하고 싶다면?
 - 매번 함수를 새로 만들 수는 없음

```
def add(a, b):  
    return a + b  
  
print(add(10, 20))
```

```
def add(*args):  
    return sum(args)  
  
print(add(10, 20))           # 30  
  
add(10, 20, 30)              # 60  
add(10, 20, 30, 40)         # 100  
add(10, 20, 30, 40, 50)     # 150
```


가변 인자

- 꼭 args라는 이름을 써야 하는 것은 아님
 - 하지만 실무에서는 *args라는 이름을 많이 사용
- 튜플로 반환

```
def print_profile(name, age, *languages):  
    print(name)  
    print(age)  
    print(languages)  
    for language in languages:  
        print(language)
```

```
print_profile('Jay', 30, 'Python', 'JavaScript', 'C++')
```

가변 인자

- `**kwargs` : 여러 개의 키워드 인자를 받을 때 사용
- 딕셔너리로 반환

```
def print_user_info(**kwargs):  
    print(kwargs)  
  
print_user_info(name="Jay", age=30, job="Student")  
# {'name': 'Jay', 'age': 30, 'job': 'Student'}
```

가변 인자

- `**kwargs` : 여러 개의 키워드 인자를 받을 때 사용
- 딕셔너리로 반환 → `get()` 함수로 안전하게 조회

```
def print_user_info(**kwargs):  
    name = kwargs.get("name")  
    age = kwargs.get("age")  
    job = kwargs.get("job")  
  
    print(f"이름: {name}")  
    print(f"나이: {age}")  
    print(f"직업: {job}")  
  
print_user_info(name="Jay", age=30, job="Student")
```

가변 인자

- Asterisk *

- 함수를 정의할 때 *를 붙이면 여러 인자를 받을 수 있음

```
def add(*args):  
    return sum(args)  
  
print(add(10, 2, 3))    # 15
```

```
print(*objects, sep=' ', end='\n', file=None, flush=False)
```

Print *objects* to the text stream *file*, separated by *sep* and followed by *end*. *sep*, *end*, *file*, and *flush*, if present, must be given as keyword arguments.

모든 비 키워드 인자는 [str\(\)](#) 이 하듯이 문자열로 변환된 후 스트림에 쓰이는데, *sep* 로 구분되고 *end* 를 뒤에 붙입니다. *sep* 과 *end* 는 모두 문자열이어야 합니다; *None* 일 수도 있는데, 기본값을 사용한다는 뜻입니다. *objects* 가 주어지지 않으면 [print\(\)](#) 는 *end* 만 씁니다.

file 인자는 `write(string)` 메서드를 가진 객체여야 합니다; 존재하지 않거나 *None* 이면, [sys.stdout](#) 이 사용됩니다. 인쇄된 인자는 텍스트 문자열로 변환되기 때문에, [print\(\)](#) 는 바이너리 모드 파일 객체와 함께 사용할 수 없습니다. 이를 위해서는. 대신 `file.write(...)` 를 사용합니다.

Output buffering is usually determined by *file*. However, if *flush* is true, the stream is forcibly flushed.

가변 인자

- Asterisk *

2. 딕셔너리를 키워드 인자로 펼칠 때 ** 사용

```
def introduce(name, age, job):  
    print(f"이름: {name}")  
    print(f"나이: {age}")  
    print(f"직업: {job}")  
  
user = {  
    "name": "Jay",  
    "age": 30,  
    "job": "Student"  
}  
  
introduce(**user)
```

가변 인자

- Asterisk *

3. 컨테이너 언패킹 시 * 사용

```
numbers = [1, 2, 3, 4, 5]

a, b, c, d, e = numbers

print(a, b, c, d, e) # 1 2 3 4 5

a, b, *rest = numbers

print(a, b, rest) # 1 2 [3, 4, 5]
```

2-3 Scope

Scope

- Python 함수는 코드 내부와 외부 구분
 - (함수 내부 : local scope / 외부 : gobal scope)
- 변수
 - 전역 변수 (global variable) : global scope(함수 외부)에 정의된 변수
 - 지역 변수 (local variable) : local scope(함수 내부)에 정의된 변수

```
message = '결과 : '
```

```
def add(a, b):  
    result = a + b  
    return result
```


Scope

- 지역 변수와 전역 변수
 - 함수 안에서 만든 변수는 보통 함수 안에서만 사용 가능
 - `result`는 함수 안에서 만든 변수 → 함수 밖에서는 사용할 수 없음 (지역 변수)

```
def add(a, b):  
    result = a + b  
    return result
```

```
print(result) # NameError: name 'result' is not defined
```

Scope

- 지역 변수와 전역 변수
 - 전역 변수 : 함수 밖에서 만든 변수
 - 함수 안에서 읽을 수 있음

```
message = '결과 : '  
  
def add(a, b):  
    result = a + b  
    return message + str(result)  
  
print(add(5, 3))
```

Scope

- 지역 변수와 전역 변수
 - `global` : 함수 안에서 전역 변수를 수정할 때 사용하는 키워드

```
message = '결과 : '  
  
def add(a, b):  
    global message  
    message = '덧셈 결과 : '  
    result = a + b  
    return message + str(result)  
  
print(add(5, 3))
```

1-4 함수 응용

함수 응용

- docstring
 - 함수에 남기는 설명 문장
 - 함수가 무엇을 하는지
 - 어떤 값을 받는지
 - 무엇을 반환하는지
 - 함수 바로 아래에 삼중 따옴표로 작성

```
def add(a, b):  
    """  
    두 숫자를 더한 결과를 반환한다.  
  
    Args :  
    a : float, int  
    b : float, int  
  
    Returns :  
    float, int : a와 b의 합  
    """  
    return a + b  
  
print(add.__doc__)
```

함수 응용

- docstring
 - 주석 : 코드 중간에 설명을 남길 때 사용
 - docstring : 함수, 클래스, 모듈의 설명을 남길 때 사용
 - 향후 Agent 개발 시 AI가 해당 함수를 호출할 때 참고 자료로 사용

```
def get_weather(location: str) -> str:
    """
    특정 지역의 날씨 정보 제공
    Args:
        location: 날씨 정보를 알고 싶은 지역의 이름 (str)
    Returns:
        해당 지역의 날씨 정보 (str)
    """
    return f"{location}의 날씨는 : 눈, -2°C, 미세먼지 매우나쁨"
```

함수 응용

- type hinting
 - 함수가 어떤 자료형을 받고, 어떤 자료형을 반환하는지 자료형을 힌트로 적어주는 문법
 - 코드 실행을 강제로 막는 기능 x
 - 사람, IDE, AI가 코드를 이해하는 데 도움

```
def get_weather(location: str) -> str:
    """
    특정 지역의 날씨 정보 제공
    Args:
    location: 날씨 정보를 알고 싶은 지역의 이름 (str)
    Returns:
    해당 지역의 날씨 정보 (str)
    """
    return f"{location}의 날씨는 : 눈, -2°C, 미세먼지 매우나쁨"
```

함수 응용

- type hinting
 - 코드 실행을 강제로 막는 기능 x

```
def add(a: int, b: int) -> int:  
    return a + b
```

```
print(add("10", "20")) # 1020
```

👉 타입이 다르더라도 에러 발생 x

함수 응용

- type hinting

자료형	type hint
정수	int
실수	float
문자열	str
Boolean	bool
리스트	list
정수 리스트	list[int]
딕셔너리	dict[str, int]

1-5 내장 함수

내장 함수

- 함수의 종류
 - 내장 함수 : 파이썬에 기본적으로 포함된 함수
 - 외장 함수 : 외부 라이브러리에서 제공하는 함수 (import를 통해 사용)
 - 사용자 정의 함수 : 사용자가 직접 작성한 함수

파이썬 내장 함수

(<https://docs.python.org/ko/3/library/functions.html>)

A abs() aiter() all() anext() any() ascii()	E enumerate() eval() exec()	L len() list() locals()	R range() repr() reversed() round()
B bin() bool() breakpoint() bytearray() bytes()	F filter() float() format() frozenset()	M map() max() memoryview() min()	S set() setattr() slice() sorted() staticmethod() str() sum() super()
C callable() chr() classmethod() compile() complex()	G getattr() globals()	N next()	T tuple() type()
D delattr() dict() dir() divmod()	H hasattr() hash() help() hex()	O object() oct() open() ord()	V vars()
	I id() input() int() isinstance() issubclass() iter()	P pow() print() property()	Z zip()
			_ __import__()

내장 함수

- `map()`
 - 여러 데이터에 같은 함수를 한 번씩 적용할 때 사용
 - `map(함수, 반복 가능한 객체)`
 - `map()`의 결과는 바로 리스트 x, 필요하면 `list()`로 감싸서 확인

```
numbers = [1, 2, 3, 4, 5]
result = []
for number in numbers:
    result.append(number ** 2)
print(result)
```



```
def square(number):
    return number ** 2

numbers = [1, 2, 3, 4, 5]
result = list(map(square, numbers))
print(result)
```

내장 함수

- `filter()`
 - 여러 데이터 중 조건을 만족하는 값만 고를 때 사용
 - `filter(함수, 반복 가능한 객체)`

```
numbers = [1, 2, 3, 4, 5]

even_numbers = []

for number in numbers:
    if number % 2 == 0:
        even_numbers.append(number)

print(even_numbers)
```



```
def is_even(number):
    return number % 2 == 0

numbers = [1, 2, 3, 4, 5]

result = list(filter(is_even, numbers))

print(result)
```

1-6 lambda

lambda

- lambda : 짧게 쓰는 이름 없는 함수
 - 일반 함수처럼 def를 쓰지 않음 (보통 한 줄로 간단한 기능을 표현할 때 사용)

```
def add(a, b):  
    return a + b
```

→

```
add = lambda a, b: a + b
```

lambda

- lambda : 짧게 쓰는 이름 없는 함수
 - 일반 함수처럼 def를 쓰지 않음 (보통 한 줄로 간단한 기능을 표현할 때 사용)

lambda 매개변수 1, 매개변수 2 : 반환값

```
add = lambda a, b: a + b
```


lambda

- lambda와 내장함수
 - 데이터 전처리 시 자주 lambda와 내장함수를 활용
- map()

```
def square(num):  
    return num ** 2  
  
numbers = [1, 2, 3, 4, 5]  
  
result = list(map(square, numbers))  
  
print(result)
```

```
result = list(map(lambda num : num ** 2, numbers))  
  
print(result)
```

lambda

- lambda와 내장함수
 - 데이터 전처리 시 자주 lambda와 내장함수를 활용
- filter()

```
def is_even(num):  
    return num % 2 == 0  
  
numbers = [1, 2, 3, 4, 5]  
  
result = list(filter(is_even, numbers))  
  
print(result)
```

```
result = list(filter(lambda x: is_even(x), numbers))  
  
print(result)
```

수고하셨습니다.

